



```
In [ ]: PAWAR SAKSHI MAHENDRA  
        ROLL NO - 49  
        PRACTICAL NO - 08
```

```
In [1]: import numpy as np  
  
        # Define sigmoid activation function  
        def sigmoid(x):  
            return 1 / (1 + np.exp(-x))  
  
        # Define derivative of sigmoid function  
        def sigmoid_derivative(x):  
            return x * (1 - x)  
  
        # Define input dataset (XOR inputs)  
        X = np.array([[0, 0],  
                       [0, 1],  
                       [1, 0],  
                       [1, 1]])  
  
        # Define output dataset (XOR outputs)  
        y = np.array([[0],  
                       [1],  
                       [1],  
                       [0]])  
  
        # Hyperparameters  
        learning_rate = 0.1  
        num_epochs = 100000  
  
        # Initialize weights randomly  
        np.random.seed(1)  
        hidden_weights = 2 * np.random.random((2, 2)) - 1  
        output_weights = 2 * np.random.random((2, 1)) - 1  
  
        # Training loop  
        for i in range(num_epochs):  
  
            # Forward propagation  
            hidden_layer = sigmoid(np.dot(X, hidden_weights))  
            output_layer = sigmoid(np.dot(hidden_layer, output_weights))  
  
            # Backpropagation  
            output_error = y - output_layer  
            output_delta = output_error * sigmoid_derivative(output_layer)  
  
            hidden_error = output_delta.dot(output_weights.T)  
            hidden_delta = hidden_error * sigmoid_derivative(hidden_layer)  
  
            # Update weights  
            output_weights += hidden_layer.T.dot(output_delta) * learning_rate  
            hidden_weights += X.T.dot(hidden_delta) * learning_rate
```

```
# Display results
print("Input:")
print(X)

print("Output after training:")
print(output_layer)
```

Input:

```
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
```

Output after training:

```
[[0.04336558]
 [0.86002574]
 [0.86003271]
 [0.55139481]]
```

In []: